

Programación Dinámica

TTPS: Técnicas y Estrategias para la Resolución de Problemas

Matías Fluxa

Universidad Nacional de La Plata

7 de septiembre de 2026

- ➊ Definición de Programación Dinámica
- ➋ Problema de Ejemplo
- ➌ Conceptos claves de la Programación Dinámica
- ➍ Técnicas de implementación
- ➎ Problemas clásicos de DP
 - LIS: Longest Increasing Subsequence
 - LCS: Longest Common Subsequence

- 1 Definición de Programación Dinámica
- 2 Problema de Ejemplo
- 3 Conceptos claves de la Programación Dinámica
- 4 Técnicas de implementación
- 5 Problemas clásicos de DP
 - LIS: Longest Increasing Subsequence
 - LCS: Longest Common Subsequence

Wikipedia

En informática, la programación dinámica es un método para reducir el tiempo de ejecución de un algoritmo mediante la utilización de subproblemas superpuestos y subestructuras óptimas.

Definiciones de distintas fuentes

Wikipedia

En informática, la programación dinámica es un método para reducir el tiempo de ejecución de un algoritmo mediante la utilización de subproblemas superpuestos y subestructuras óptimas.

Gemini

La programación dinámica es una **técnica** de optimización matemática y un método de diseño de algoritmos que resuelve problemas complejos dividiéndolos en subproblemas más pequeños y sencillos. La clave de esta técnica es que almacena las soluciones de estos subproblemas para no tener que volver a calcularlas más tarde.

- ① Definición de Programación Dinámica
- ② Problema de Ejemplo
- ③ Conceptos claves de la Programación Dinámica
- ④ Técnicas de implementación
- ⑤ Problemas clásicos de DP
 - LIS: Longest Increasing Subsequence
 - LCS: Longest Common Subsequence

Un entero positivo N se denomina fachero si cumple las siguientes condiciones:

- Tiene 8 dígitos
- Todos los dígitos de N son impares
- Todas las restas de dos dígitos consecutivos de N (el mayor menos el menor) son iguales a 2.

Hallar la cantidad de números facheros que existen.

Resolución del ejercicio

- Podemos pensar en un principio en definir:
 $f(x)$: Cantidad de números facheros de x dígitos.

Resolución del ejercicio

- Podemos pensar en un principio en definir:
 $f(x)$: Cantidad de números facheros de x dígitos.
- Nuestro problema nos pide específicamente hallar $f(8)$.

Resolución del ejercicio

- Podemos pensar en un principio en definir:
 $f(x)$: Cantidad de números facheros de x dígitos.
- Nuestro problema nos pide específicamente hallar $f(8)$.
- Si queremos seguir con la definición de DP que vimos anteriormente, podríamos pensar en resolver casos más chicos primero.

Resolución del ejercicio

- Podemos pensar en un principio en definir:
 $f(x)$: Cantidad de números facheros de x dígitos.
- Nuestro problema nos pide específicamente hallar $f(8)$.
- Si queremos seguir con la definición de DP que vimos anteriormente, podríamos pensar en resolver casos más chicos primero.
- Supongamos que logramos hallar $f(7)$ de alguna manera, ¿podríamos obtener fácilmente $f(8)$ a partir del resultado conocido?

Resolución del ejercicio

- La respuesta es que no es tan fácil obtener $f(8)$ a partir de $f(7)$, ya que necesitamos conocer el último dígito del número de 7 dígitos para poder determinar cuántos números facheros de 8 dígitos se pueden formar a partir de él.

Resolución del ejercicio

- La respuesta es que no es tan fácil obtener $f(8)$ a partir de $f(7)$, ya que necesitamos conocer el último dígito del número de 7 dígitos para poder determinar cuántos números fachersos de 8 dígitos se pueden formar a partir de él.
- Si por ejemplo, un número fachero de 7 dígitos termina en 3, el siguiente dígito del número fachero de 8 dígitos puede ser 1 o 5, pero si es 9, el siguiente dígito solo puede ser 7.

Resolución del ejercicio

- La respuesta es que no es tan fácil obtener $f(8)$ a partir de $f(7)$, ya que necesitamos conocer el último dígito del número de 7 dígitos para poder determinar cuántos números fachersos de 8 dígitos se pueden formar a partir de él.
- Si por ejemplo, un número fachero de 7 dígitos termina en 3, el siguiente dígito del número fachero de 8 dígitos puede ser 1 o 5, pero si es 9, el siguiente dígito solo puede ser 7.
- Por lo tanto, para poder aplicar programación dinámica de manera efectiva, necesitamos redefinir nuestro **estado** no solo en función de la cantidad de dígitos, sino también del último dígito del número fachero.

Resolución del ejercicio

- Conviene cambiar la definición del estado de la siguiente manera:

$g(x, k)$: # de números facheros de x dígitos que terminan con el dígito k .

Resolución del ejercicio

- Conviene cambiar la definición del estado de la siguiente manera:

$g(x, k)$: # de números facheros de x dígitos que terminan con el dígito k .

- Notemos que:

$$f(8) = g(8, 1) + g(8, 3) + g(8, 5) + g(8, 7) + g(8, 9)$$

Resolución del ejercicio

- Conviene cambiar la definición del estado de la siguiente manera:

$g(x, k)$: # de números facheros de x dígitos que terminan con el dígito k .

- Notemos que:

$$f(8) = g(8, 1) + g(8, 3) + g(8, 5) + g(8, 7) + g(8, 9)$$

- Sabiendo todos los $g(7, k)$, veamos que ahora es más fácil obtener todos $g(8, k)$.

Resolución del ejercicio

$$g(7, 1)$$

$$g(8, 1)$$

$$g(7, 3)$$

$$g(8, 3)$$

$$g(7, 5)$$

$$g(8, 5)$$

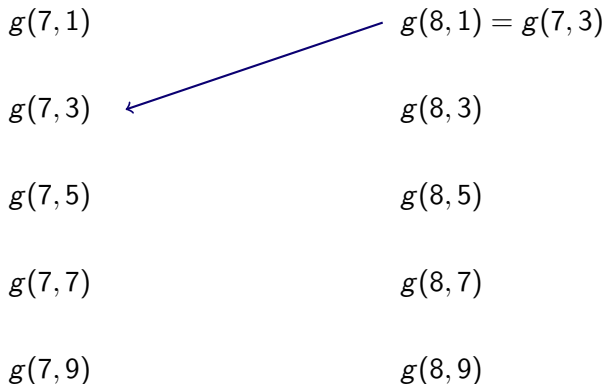
$$g(7, 7)$$

$$g(8, 7)$$

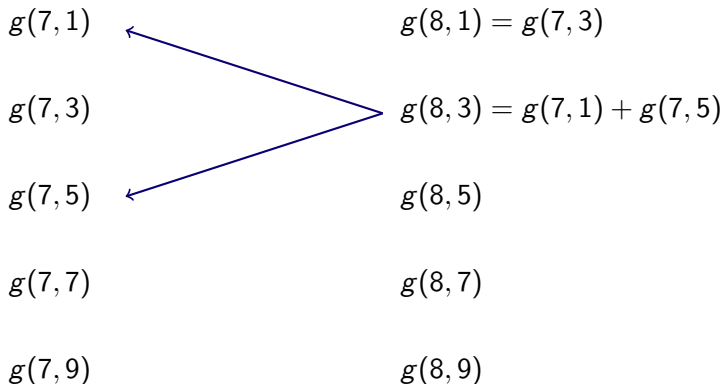
$$g(7, 9)$$

$$g(8, 9)$$

Resolución del ejercicio



Resolución del ejercicio



Resolución del ejercicio

$$g(7, 1)$$

$$g(8, 1) = g(7, 3)$$

$$g(7, 3)$$

$$g(8, 3) = g(7, 1) + g(7, 5)$$

$$g(7, 5)$$

$$g(8, 5) = g(7, 3) + g(7, 7)$$

$$g(7, 7)$$

$$g(8, 7)$$

$$g(7, 9)$$

$$g(8, 9)$$

Resolución del ejercicio

$$g(7, 1)$$

$$g(8, 1) = g(7, 3)$$

$$g(7, 3)$$

$$g(8, 3) = g(7, 1) + g(7, 5)$$

$$g(7, 5)$$

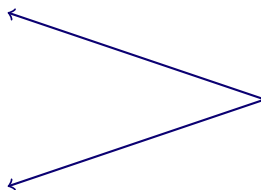
$$g(8, 5) = g(7, 3) + g(7, 7)$$

$$g(7, 7)$$

$$g(8, 7) = g(7, 5) + g(7, 9)$$

$$g(7, 9)$$

$$g(8, 9)$$



Resolución del ejercicio

$$g(7, 1)$$

$$g(8, 1) = g(7, 3)$$

$$g(7, 3)$$

$$g(8, 3) = g(7, 1) + g(7, 5)$$

$$g(7, 5)$$

$$g(8, 5) = g(7, 3) + g(7, 7)$$

$$g(7, 7)$$

$$g(8, 7) = g(7, 5) + g(7, 9)$$

$$g(7, 9)$$

$$g(8, 9) = g(7, 7)$$



- Como aún no sabemos la cantidad de números facheros que existen de 7 dígitos, debemos hacer el mismo proceso en el paso:

7 dígitos $\rightarrow$ 6 dígitos

Resolución del ejercicio

- Como aún no sabemos la cantidad de números facteros que existen de 7 dígitos, debemos hacer el mismo proceso en el paso:

7 dígitos $\rightarrow$ 6 dígitos

- Recursivamente, vamos a ir reduciendo la cantidad de dígitos, hasta llegar a un caso base o que sea trivial de resolver, que en este caso puede ser de un dígito.

Resolución del ejercicio

- Como aún no sabemos la cantidad de números facheros que existen de 7 dígitos, debemos hacer el mismo proceso en el paso:

7 dígitos $\rightarrow$ 6 dígitos

- Recursivamente, vamos a ir reduciendo la cantidad de dígitos, hasta llegar a un caso base o que sea trivial de resolver, que en este caso puede ser de un dígito.
- Es fácil ver que:

$$g(1, 1) = g(1, 3) = g(1, 5) = g(1, 7) = g(1, 9) = 1$$

Obtendríamos una tabla como la siguiente:

Termina en ...	# Dígitos							
	1	2	3	4	5	6	7	8
1	1	1	2	3	6	9	18	27
3	1	2	3	6	9	18	27	54
5	1	2	4	6	12	18	36	54
7	1	2	3	6	9	18	27	54
9	1	1	2	3	6	9	18	27
Total	5	8	14	24	42	72	126	216

Obtendríamos una tabla como la siguiente:

Termina en ...	# Dígitos							
	1	2	3	4	5	6	7	8
1	1	1	2	3	6	9	18	27
3	1	2	3	6	9	18	27	54
5	1	2	4	6	12	18	36	54
7	1	2	3	6	9	18	27	54
9	1	1	2	3	6	9	18	27
Total	5	8	14	24	42	72	126	216

Por lo tanto, la cantidad de números facheros de 8 dígitos es **216**.

- 1 Definición de Programación Dinámica
- 2 Problema de Ejemplo
- 3 Conceptos claves de la Programación Dinámica
- 4 Técnicas de implementación
- 5 Problemas clásicos de DP
 - LIS: Longest Increasing Subsequence
 - LCS: Longest Common Subsequence

¿Qué es finalmente la Programación Dinámica?

Definición conceptual - By Monazo

La programación dinámica es una **técnica** de resolución de problemas mediante la definición de **estados** y la obtención de información de ellos a través de sus **relaciones** con otros estados más simples.

¿Cuánto aplicar la programación dinámica?

La programación dinámica es conveniente aplicarla cuando:

- Podemos definir un estado a partir de las características del problema.

¿Cuánto aplicar la programación dinámica?

La programación dinámica es conveniente aplicarla cuando:

- Podemos definir un estado a partir de las características del problema.
- Podemos conocer fácilmente la dependencia de los estados entre sí (conocer las relaciones).

¿Cuánto aplicar la programación dinámica?

La programación dinámica es conveniente aplicarla cuando:

- Podemos definir un estado a partir de las características del problema.
- Podemos conocer fácilmente la dependencia de los estados entre sí (conocer las relaciones).
- Debido a que cada estado y cada relación debe ser procesada exactamente una vez, la cantidad de estados sumada a la cantidad de relaciones debe ser un número que se adecue a nuestras limitaciones de tiempo y memoria.

¿Cuánto aplicar la programación dinámica?

La programación dinámica es conveniente aplicarla cuando:

- Podemos definir un estado a partir de las características del problema.
- Podemos conocer fácilmente la dependencia de los estados entre sí (conocer las relaciones).
- Debido a que cada estado y cada relación debe ser procesada exactamente una vez, la cantidad de estados sumada a la cantidad de relaciones debe ser un número que se adecue a nuestras limitaciones de tiempo y memoria.
- Las relaciones tienden a llegar a los casos bases, es decir, no se generan “bucles” entre las relaciones. En palabras más técnicas, los estados y las relaciones forman un **Grafo Dirigido Acíclico (DAG)**.

- 1 Definición de Programación Dinámica
- 2 Problema de Ejemplo
- 3 Conceptos claves de la Programación Dinámica
- 4 Técnicas de implementación
- 5 Problemas clásicos de DP
 - LIS: Longest Increasing Subsequence
 - LCS: Longest Common Subsequence

Técnicas de implementación

Existen principalmente dos técnicas para implementar las transiciones entre estados:

- **Bottom-Up:** Consiste en partir de los casos bases, e ir "propagando" la respuesta hasta llegar al problema principal que era de interés. Sus implementaciones suelen ser **iterativas**.
- **Top-Down:** Consiste en partir desde el problema principal, e ir dividiéndolo en problemas más pequeños hasta llegar a los casos bases. Sus implementaciones suelen ser **recursivas**. Para evitar visitar múltiples veces los mismos estados, suelen implementarse con **Memoization**.

No hay un método que siempre sea mejor que el otro, su elección suele depender muchas veces del problema a resolver y cuál facilita su implementación.

Ejemplo de Bottom-Up

Vamos a hallar el n -ésimo término de Fibonacci con Bottom-Up. Recordemos que la sucesión de Fibonacci cumple que cada elemento es igual a la suma de los dos anteriores, es decir:

$$f(i) = f(i - 1) + f(i - 2)$$

Ejemplo de Bottom-Up

Vamos a hallar el n -ésimo término de Fibonacci con Bottom-Up. Recordemos que la sucesión de Fibonacci cumple que cada elemento es igual a la suma de los dos anteriores, es decir:

$$f(i) = f(i - 1) + f(i - 2)$$

- Para el caso Bottom-Up, arrancamos definiendo los casos bases $f(0) = 0$ y $f(1) = 1$.

Ejemplo de Bottom-Up

Vamos a hallar el n -ésimo término de Fibonacci con Bottom-Up. Recordemos que la sucesión de Fibonacci cumple que cada elemento es igual a la suma de los dos anteriores, es decir:

$$f(i) = f(i - 1) + f(i - 2)$$

- Para el caso Bottom-Up, arrancamos definiendo los casos bases $f(0) = 0$ y $f(1) = 1$.
- Iteramos i desde 2 a n , y en cada paso hallamos $f(i)$.

Ejemplo de Bottom-Up

Vamos a hallar el n -ésimo término de Fibonacci con Bottom-Up. Recordemos que la sucesión de Fibonacci cumple que cada elemento es igual a la suma de los dos anteriores, es decir:

$$f(i) = f(i - 1) + f(i - 2)$$

- Para el caso Bottom-Up, arrancamos definiendo los casos bases $f(0) = 0$ y $f(1) = 1$.
- Iteramos i desde 2 a n , y en cada paso hallamos $f(i)$.
- De esta manera, vamos resolviendo desde los casos más chicos, hasta llegar al problema original.

Ejemplo de Bottom-Up

Implementación en C++

```
int fibBottomUp(int n) {  
    vector<int> fib(n + 5);  
    fib[0] = 0;  
    fib[1] = 1;  
    for (int i = 2; i <= n; i++) {  
        fib[i] = fib[i - 1] + fib[i - 2];  
    }  
    return fib[n];  
}
```

Ejemplo de Top-Down

Ahora, vamos a hallar el n -ésimo término de Fibonacci con Top-Down.

- Para este caso, comenzamos desde el problema principal. Sabemos que:

$$f(n) = f(n-1) + f(n-2)$$

- Como aún no sabemos ninguno de los otros dos, resolvemos recursivamente $f(n-1)$:

$$f(n-1) = f(n-2) + f(n-3)$$

- Y así recursivamente hasta llegar a los casos bases:

$$f(2) = f(1) + f(0)$$

Ejemplo de Top-Down

Implementación en C++

```
int fibTopDown(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    if (n == 1) {  
        return 1;  
    }  
    return fibTopDown(n - 1) + fibTopDown(n - 2);  
}
```

Ejemplo de Top-Down

Implementación en C++

WARNING

Esta implementación puede generar que la recursión visite más de una vez cada estado.

Para el caso de Fibonacci, se hace exponencial el tiempo de ejecución. Por eso se implementa con **Memoization**.

```
int fibTopDown(int n) {  
    if (n == 0) {  
        return 0;  
    }  
    if (n == 1) {  
        return 1;  
    }  
    return fibTopDown(n - 1) +  
        fibTopDown(n - 2);  
}
```

Ejemplo de Top-Down

Implementación en C++

Con Memoization

El concepto consiste en pagar memoria para reducir ejecuciones redundantes.

```
vector<int> memo;

int fibTopDown(int n) {
    if (memo[n] != -1) return memo[
        n];
    if (n == 0) {
        return 0;
    }
    if (n == 1) {
        return 1;
    }
    memo[n] = fibTopDown(n - 1) +
        fibTopDown(n - 2);
    return memo[n];
}
```

- 1 Definición de Programación Dinámica
- 2 Problema de Ejemplo
- 3 Conceptos claves de la Programación Dinámica
- 4 Técnicas de implementación
- 5 Problemas clásicos de DP
 - LIS: Longest Increasing Subsequence
 - LCS: Longest Common Subsequence

Longest Increasing Subsequence (LIS)

Dado un arreglo de n números, hallar la subsecuencia creciente más grande posible.

Una subsecuencia es una secuencia que se obtiene del arreglo original, eliminando algunos de sus elementos (posiblemente cero) y conservando su orden original.

Para que sea creciente, cada número de la secuencia debe ser estrictamente mayor que el anterior.

Por ejemplo:

$$A = (0, 8, 4, 12, 2, 10, 6, 14, 1, 9, 5, 13, 3, 11, 7, 15)$$

Una de las subsecuencias crecientes más largas es:

$$A' = (0, 2, 6, 9, 11, 15)$$

Longest Increasing Subsequence (LIS)

Solución

Definimos nuestro estado:

- **dp[i]**: La longitud de la subsecuencia más larga que termina en el i -ésimo elemento del arreglo.

Inicialmente, **dp[i]** = 1 $(\forall i \in [1, 2, \dots, n])$.

Luego las dependencias quedan:

$$\mathbf{dp[i]} = \text{máx}(\mathbf{dp[j]} + 1) \quad (\forall j < i : a[j] < a[i])$$

La respuesta final sería $\text{máx}(\mathbf{dp[i]}) \quad (\forall i \in [1, 2, \dots, n])$.

Longest Increasing Subsequence (LIS)

Solución

- Complejidad en memoria: $\mathcal{O}(n)$
- Complejidad en tiempo de ejecución: $\mathcal{O}(n^2)$

Desafío

- Implementar la solución con tiempo de ejecución $\mathcal{O}(n \cdot \log(n))$.

Longest Common Subsequence (LCS)

Dado un arreglo **a** de **n** números y un arreglo **b** de **m** números, hallar la mayor subsecuencia en común a ambos arreglos.

Por ejemplo:

$$a = (3, 1, 3, 2, 7, 4, 8, 2)$$

$$b = (6, 5, 1, 2, 3, 4)$$

Entonces una posible mayor subsecuencia en común es la secuencia (1, 2, 4).

Longest Common Subsequence (LCS)

Solución

Definimos nuestro estado de la siguiente manera:

- $\mathbf{dp}[i][j]$: La mayor subsecuencia en común considerando los primeros i elementos del arreglo a y los primeros j elementos del arreglo b .

Inicialmente: $\mathbf{dp}[0][0] = 0$.

Luego, para cada estado (i, j) inicializamos:

$$\mathbf{dp}[i][j] = \text{máx}(\mathbf{dp}[i-1][j], \mathbf{dp}[i][j-1])$$

Si $a[i] == b[j]$, entonces:

$$\mathbf{dp}[i][j] = \text{máx}(\mathbf{dp}[i][j], \mathbf{dp}[i-1][j-1] + 1)$$

La respuesta final sería $\mathbf{dp}[n][m]$.

Longest Common Subsequence (LCS)

Solución

- Complejidad en memoria: $\mathcal{O}(n \cdot m)$ (o $\mathcal{O}(n^2)$ si $n = m$)
- Complejidad en tiempo de ejecución: $\mathcal{O}(n \cdot m)$ (o $\mathcal{O}(n^2)$ si $n = m$)

Desafío

- Implementar la solución con memoria $\mathcal{O}(\min(n, m))$ (o $\mathcal{O}(n)$).

Muchas Gracias